Ex. No. 11	Database Integration in NodeJS Application using MongoDB
Date of Exercise	13/10/2026
GitHub Host Link	https://github.com/Manisha-R2405/web-tech.git

Aim

To develop a NodeJS application that connects to a MongoDB database and performs basic database operations such as Create, Read, Update and Delete (CRUD).

Description

Software Requirements:

- Node.js
- npm
- Express.js
- MongoDB
- Mongoose
- Visual Studio Code
- Web browser or Postman

Node.js is a JavaScript runtime environment used to develop server-side applications. MongoDB is a NoSQL database that stores data in the form of documents. Mongoose is an Object Data Modeling (ODM) library used to connect Node.js applications with MongoDB and perform database operations.

In this experiment, a Node.js application is developed using Express.js and integrated with the MongoDB database named `studentdb`. A Student Schema is defined using Mongoose to specify student details such as register number, name, department, email, and CGPA. A Mongoose Model is created to perform database operations on student records.

The application implements CRUD operations using HTTP methods. The POST method is used to insert student records, GET is used to retrieve all students or a particular student, PUT is used to update student details, and DELETE is used to remove a student record. The application is tested using PowerShell commands in the VS Code terminal, and the database and collection are verified using MongoDB Compass.

Thus, this experiment demonstrates the integration of Node.js with MongoDB and the implementation of basic database operations in a web application.

Procedure

1. Created a project folder named exp11 in VS Code.
2. Initialized the Node.js project using npm init -y.
3. Installed Express and Mongoose using npm install express mongoose.
4. Started the MongoDB service and connected to the studentdb database.
5. Created app.js with the Student Schema and Mongoose Model.
6. Implemented API routes to insert, retrieve, update and delete student records.
7. Tested the API routes using PowerShell commands in the VS Code terminal.
8. Verified the successful execution of CRUD operations.

Sample Questions

1. Create a New Project Folder

Create a folder named:

NodeMongoLab

Open the folder in Visual Studio Code. Open the integrated terminal and run:

npm init -y

This command creates the package.json file.

2. Install Required Packages

Install Express and Mongoose using the following command:

npm install express mongoose

For automatic server restart during development, you may optionally install Nodemon:

npm install --save-dev nodemon

The initial project structure will be:

NodeMongoLab/

├── node_modules/

├── package.json

└── app.js

3. Start MongoDB

If MongoDB is installed locally, start the MongoDB service. The default MongoDB connection URL is usually:

mongodb://127.0.0.1:27017

For this experiment, create/use the database named studentdb. The connection string becomes:

mongodb://127.0.0.1:27017/studentdb

MongoDB will create the database automatically when the first record is inserted.

4. Create app.js and Connect NodeJS with MongoDB

Create a file named app.js and enter the following code:

const express = require("express");

const mongoose = require("mongoose");

const app = express();

app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/studentdb")

.then(() => console.log("MongoDB Connected Successfully"))

.catch((error) => console.log("Database Connection Error:", error));

app.get("/", (req, res) => {

res.send("NodeJS MongoDB Application is Running");

});

app.listen(3000, () => {

console.log("Server running at http://localhost:3000");

});

Run the application using:

node app.js

Expected terminal output:

Server running at http://localhost:3000

MongoDB Connected Successfully

Open http://localhost:3000 in a browser. The page should display:

NodeJS MongoDB Application is Running

5. Create a Student Schema

MongoDB stores information as documents. Using Mongoose, define the structure of the student document with a

schema:

const studentSchema = new mongoose.Schema({

regNo: { type: String, required: true },

name: { type: String, required: true },

department: { type: String, required: true },

email: { type: String },

cgpa: { type: Number }

});

6. Create a Mongoose Model

Create a model using the schema:

const Student = mongoose.model("Student", studentSchema);

Mongoose will automatically create a MongoDB collection named students.

7. CREATE Operation – Insert a Student Record

Add the following POST route:

app.post("/students", async (req, res) => {

try {

const student = new Student({

regNo: req.body.regNo,

name: req.body.name,

department: req.body.department,

email: req.body.email,

cgpa: req.body.cgpa

});

const savedStudent = await student.save();

res.status(201).json(savedStudent);

} catch (error) {

res.status(500).json({ message: error.message });

}

});

Restart the server. If Nodemon is installed, run:

npx nodemon app.js

8. Insert Data Using Postman

In Postman, choose POST and use the URL:

http://localhost:3000/students

Choose Body → raw → JSON, then enter:

{

"regNo": "URK23CS1001",

"name": "Arun Kumar",

"department": "Computer Science and Engineering",

"email": "arun@gmail.com",

"cgpa": 8.5

}

Click Send. The saved record will be returned as JSON along with an automatically generated _id field.

9. READ Operation – Display All Students

app.get("/students", async (req, res) => {

try {

const students = await Student.find();

res.json(students);

} catch (error) {

```
res.status(500).json({ message: error.message });
}
});
```

Test using:

GET http://localhost:3000/students

10. Read One Student Using Register Number

```
app.get(&quot;/students/:regNo&quot;, async (req, res) =&gt; {
try {
const student = await Student.findOne({ regNo: req.params.regNo });
if (!student) {
return res.status(404).json({ message: &quot;Student not found&quot; });
}
res.json(student);
} catch (error) {
res.status(500).json({ message: error.message });
}
});
```

Example:

GET http://localhost:3000/students/URK23CS1001

11. UPDATE Operation – Modify Student Details

```
app.put(&quot;/students/:regNo&quot;, async (req, res) =&gt; {
try {
const updatedStudent = await Student.findOneAndUpdate(
```

```
{ regNo: req.params.regNo },
{
name: req.body.name,
department: req.body.department,
email: req.body.email,
cgpa: req.body.cgpa
},
{ new: true }
);
if (!updatedStudent) {
return res.status(404).json({ message: &amp;quot;Student not found&amp;quot; });
}
res.json(updatedStudent);
} catch (error) {
res.status(500).json({ message: error.message });
}
});
```

Use the following endpoint in Postman:

PUT http://localhost:3000/students/URK23CS1001

Sample JSON body:

```
{
&amp;quot;name&amp;quot;: &amp;quot;Arun Kumar&amp;quot;,
&amp;quot;department&amp;quot;: &amp;quot;CSE&amp;quot;,
&amp;quot;email&amp;quot;: &amp;quot;arun.kumar@gmail.com&amp;quot;,
```

"cgpa": 9.0

}

12. DELETE Operation – Remove a Student

app.delete("/students/:regNo", async (req, res) => {

try {

const deletedStudent = await Student.findOneAndDelete({

regNo: req.params.regNo

});

if (!deletedStudent) {

return res.status(404).json({ message: "Student not found" });

}

res.json({ message: "Student deleted successfully" });

} catch (error) {

res.status(500).json({ message: error.message });

}

});

Test with:

DELETE http://localhost:3000/students/URK23CS1001

Expected response:

{

"message": "Student deleted successfully"

}

13. Complete Program

```
const express = require(&amp;quot;express&amp;quot;);
const mongoose = require(&amp;quot;mongoose&amp;quot;);
const app = express();
app.use(express.json());
mongoose.connect(&amp;quot;mongodb://127.0.0.1:27017/studentdb&amp;quot;)
.then(() =&amp;gt; console.log(&amp;quot;MongoDB Connected Successfully&amp;quot;))
.catch((error) =&amp;gt; console.log(&amp;quot;Database Connection Error:&amp;quot;, error));
const studentSchema = new mongoose.Schema({
regNo: { type: String, required: true },
name: { type: String, required: true },
department: { type: String, required: true },
email: { type: String },
cgpa: { type: Number }
});
const Student = mongoose.model(&amp;quot;Student&amp;quot;, studentSchema);
app.get(&amp;quot;/&amp;quot;, (req, res) =&amp;gt; {
res.send(&amp;quot;NodeJS MongoDB Application is Running&amp;quot;);
});
app.post(&amp;quot;/students&amp;quot;, async (req, res) =&amp;gt; {
try {
const student = new Student(req.body);
const savedStudent = await student.save();
res.status(201).json(savedStudent);
} catch (error) {
```

```
res.status(500).json({ message: error.message });
}
});
app.get(&quot;/students&quot;, async (req, res) =&gt; {
try {
const students = await Student.find();
res.json(students);
} catch (error) {
res.status(500).json({ message: error.message });
}
});
app.get(&quot;/students/:regNo&quot;, async (req, res) =&gt; {
try {
const student = await Student.findOne({ regNo: req.params.regNo });
if (!student) return res.status(404).json({ message: &quot;Student not found&quot; });
res.json(student);
} catch (error) {
res.status(500).json({ message: error.message });
}
});
app.put(&quot;/students/:regNo&quot;, async (req, res) =&gt; {
try {
const student = await Student.findOneAndUpdate(
{ regNo: req.params.regNo },
req.body,
```

```
{ new: true }
);
if (!student) return res.status(404).json({ message: &amp;quot;Student not found&amp;quot; });
res.json(student);
} catch (error) {
res.status(500).json({ message: error.message });
}
});
app.delete(&amp;quot;/students/:regNo&amp;quot;, async (req, res) =&amp;gt; {
try {
const student = await Student.findOneAndDelete({ regNo: req.params.regNo });
if (!student) return res.status(404).json({ message: &amp;quot;Student not found&amp;quot; });
res.json({ message: &amp;quot;Student deleted successfully&amp;quot; });
} catch (error) {
res.status(500).json({ message: error.message });
}
});
app.listen(3000, () =&amp;gt; {
console.log(&amp;quot;Server running at http://localhost:3000&amp;quot;);
});
```

14. Verify the Database Using MongoDB Compass

Open MongoDB Compass and connect using:

mongodb://127.0.0.1:27017

Open the database studentdb and the collection students. A sample document may appear as:

{

"_id": "68...",

"regNo": "URK23CS1001",

"name": "Arun Kumar",

"department": "Computer Science and Engineering",

"email": "arun@gmail.com",

"cgpa": 8.5

}

15. CRUD Operations Summary

Operation HTTP Method URL

Create student POST /students

View all students GET /students

View one student GET /students/:regNo

Update student PUT /students/:regNo

Delete student DELETE /students/:regNo

16. Expected Output

Insert:

{

"regNo": "URK23CS1001",

"name": "Arun Kumar",

"department": "CSE",

"email": "arun@gmail.com",

"cgpa": 8.5

}

Read:

[

{

"regNo": "URK23CS1001",

"name": "Arun Kumar",

"department": "CSE",

"cgpa": 8.5

}

]

Update:

{

"regNo": "URK23CS1001",

"name": "Arun Kumar",

"department": "CSE",

"cgpa": 9

}

Delete:

{

"message": "Student deleted successfully"

}

Code:

App.js

```
const express = require("express");
```

```
const mongoose = require("mongoose");
const app = express();
app.use(express.json());
mongoose.connect("mongodb://127.0.0.1:27017/studentdb")
    .then(() => console.log("MongoDB Connected Successfully"))
    .catch((error) => console.log("Database Connection Error:", error));
const studentSchema = new mongoose.Schema({
    regNo: { type: String, required: true },
    name: { type: String, required: true },
    department: { type: String, required: true },
    email: { type: String },
    cgpa: { type: Number }
});
const Student = mongoose.model("Student", studentSchema);
app.get("/", (req, res) => {
    res.send("NodeJS MongoDB Application is Running");
});
app.post("/students", async (req, res) => {
    try {
        const student = new Student(req.body);
        const savedStudent = await student.save();
        res.status(201).json(savedStudent);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
```

```
});
app.get("/students", async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
app.get("/students/:regNo", async (req, res) => {
  try {
    const student = await Student.findOne({
      regNo: req.params.regNo
    });

    if (!student) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json(student);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
```

```
});


app.put("/students/:regNo", async (req, res) => {
  try {
    const updatedStudent = await Student.findOneAndUpdate(
      { regNo: req.params.regNo },
      req.body,
      { new: true, runValidators: true }
    );

    if (!updatedStudent) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json(updatedStudent);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
app.delete("/students/:regNo", async (req, res) => {
  try {
    const deletedStudent = await Student.findOneAndDelete({
      regNo: req.params.regNo
```

```
    });

    if (!deletedStudent) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json({
      message: "Student deleted successfully"
    });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Command Prompt(Terminal 1):

```
PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11> npm init -y
Wrote to C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11\package.json:

{
  "name": "exp11",
  "version": "1.0.0",
```

```
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11> npm install express mongoose

added 83 packages, and audited 84 packages in 9s

29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11> Get-Service MongoDB

Status   Name               DisplayName
------   ----               -----------
Running  MongoDB            MongoDB Server (MongoDB)

PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11> node app.js
```

```
Server running at http://localhost:3000

MongoDB Connected Successfully

PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11> node app.js

Server running at http://localhost:3000

MongoDB Connected Successfully
```

Terminal 2:

```
PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11>

>> $student = @{

>>     regNo = "URK23CS1001"

>>     name = "Arun Kumar"

>>     department = "Computer Science and Engineering"

>>     email = "arun@gmail.com"

>>     cgpa = 8.5

>> } | ConvertTo-Json

>>

>> Invoke-RestMethod -Uri "http://localhost:3000/students" -Method Post -ContentType "application/json" -Body $student

>>

regNo      : URK23CS1001

name       : Arun Kumar

department : Computer Science and Engineering

email      : arun@gmail.com

cgpa       : 8.5

_id        : 6ace38970a82826b30450daf

__v        : 0
```

```
PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11>
>> Invoke-RestMethod -Uri "http://localhost:3000/students" -Method Get
>>
_id        : 6ace38970a82826b30450daf
regNo      : URK23CS1001
name       : Arun Kumar
department : Computer Science and Engineering
email      : arun@gmail.com
cgpa       : 8.5
__v        : 0

PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11>
>> Invoke-RestMethod -Uri "http://localhost:3000/students/URK23CS1001" -Method Get
>>
_id        : 6ace38970a82826b30450daf
regNo      : URK23CS1001
name       : Arun Kumar
department : Computer Science and Engineering
email      : arun@gmail.com
cgpa       : 8.5
__v        : 0
PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11>
>> $update = @{
>>     name = "Arun Kumar"
>>     department = "CSE"
```

```
>>     email = "arun.kumar@gmail.com"

>>     cgpa = 9.0

>> } | ConvertTo-Json

>>

>> Invoke-RestMethod -Uri "http://localhost:3000/students/URK23CS1001" -Method Put -ContentType "application/json" -Body $update

>>

_id        : 6ace38970a82826b30450daf

regNo      : URK23CS1001

name       : Arun Kumar

department : CSE

email      : arun.kumar@gmail.com

cgpa       : 9

__v        : 0

PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11>

>> Invoke-RestMethod -Uri "http://localhost:3000/students/URK23CS1001" -Method Delete

>>

message

-------

Student deleted successfully

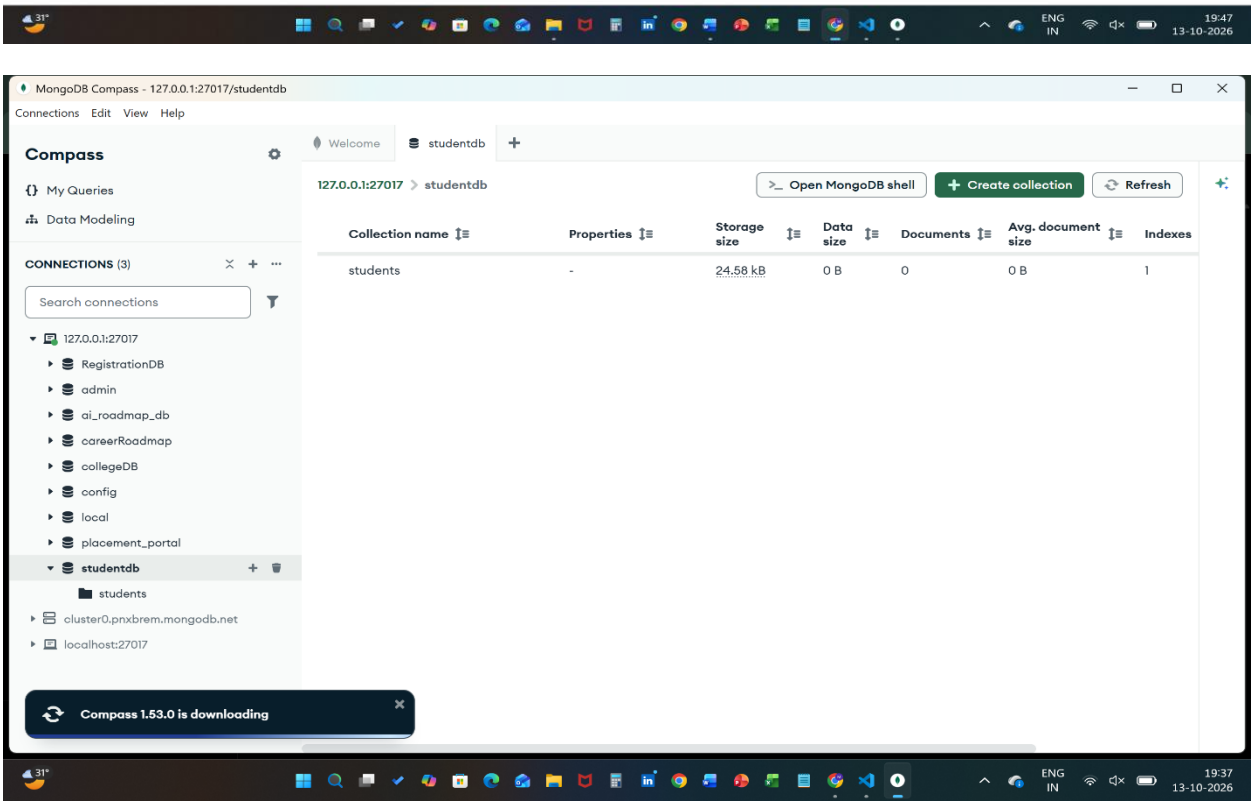
PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11>

>> Invoke-RestMethod -Uri "http://localhost:3000/students" -Method Get

>>

PS C:\Users\Manisha Rajalingam\Sem 5 Lab\WEB\exp11>
```





Result

Thus, a NodeJS application was successfully integrated with MongoDB using the Mongoose library, and CRUD operations such as insertion, retrieval, updating and deletion of student records were successfully performed.